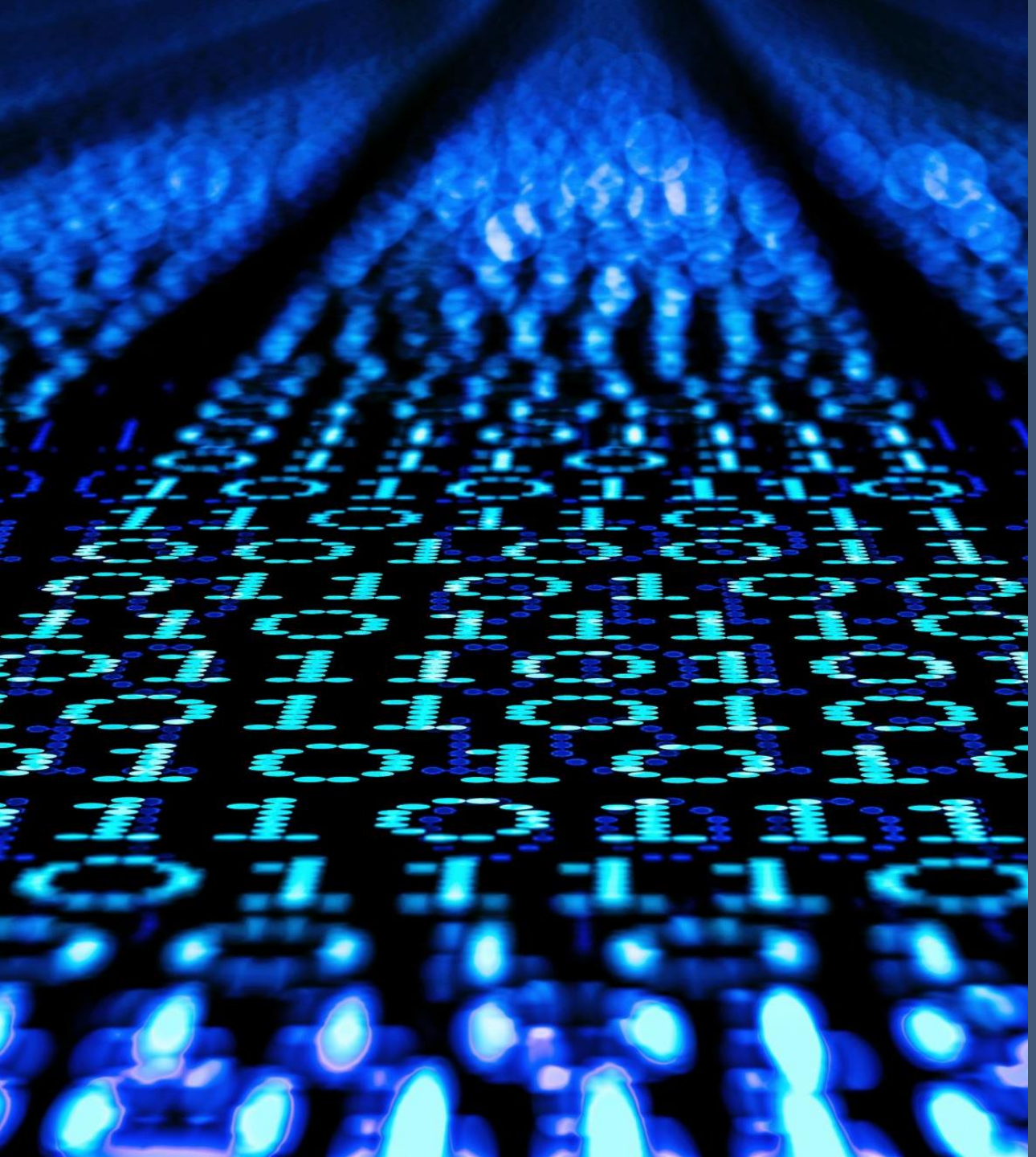


ÁRVORES BINÁRIAS

Prof. Ricardo Mesquita

PROF. RICARDO MESQUITA

ÁRVORES BINÁRIAS DE BUSCA

ÁRVORE BINÁRIA DE BUSCA

- Em uma árvore binária de busca cada nó contém um campo chamado *key*, podendo haver outras informações, além dos ponteiros *left* e *right*.
- O campo *key* especifica em geral uma chave que identifica de forma única um determinado registro ou informação.
  - Exemplos de chaves: número de identidade, número CPF, etc. Assim, suporemos que todos os valores de *key* são distintos.

PROF. RICARDO MESQUITA

ÁRVORE BINÁRIA DE BUSCA

- Dado um valor qualquer, deseja-se localizar o nó da árvore, se houver, cujo *key* é igual ao valor dado.
- Característica da árvore binária de busca: Para todo nó *x* da árvore binária de busca, as seguintes condições são verificadas.
  - *key*(x) > *key*(y), para todo nó *y* da subárvore esquerda.
  - *key*(x) < *key*(y), para todo nó *y* da subárvore direita.

PROF. RICARDO MESQUITA

ÁRVORE BINÁRIA DE BUSCA

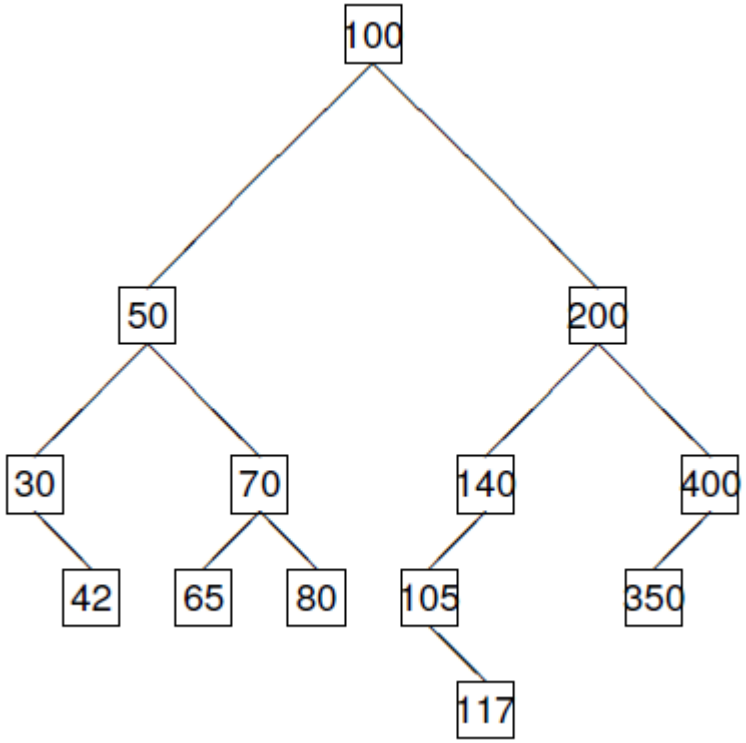
PROF. RICARDO MESQUITA

Exemplo:

- Para todo nó *x* da árvore binária de busca:
  - *key*(x) ≥ *key*(y), para todo nó *y* da subárvore esquerda.
  - *key*(x) < *key*(y), para todo nó *y* da subárvore direita.

Qual a saída do percurso *em-ordem*?

30 42 50 65 70 80 100
105 117 140 200 350 400



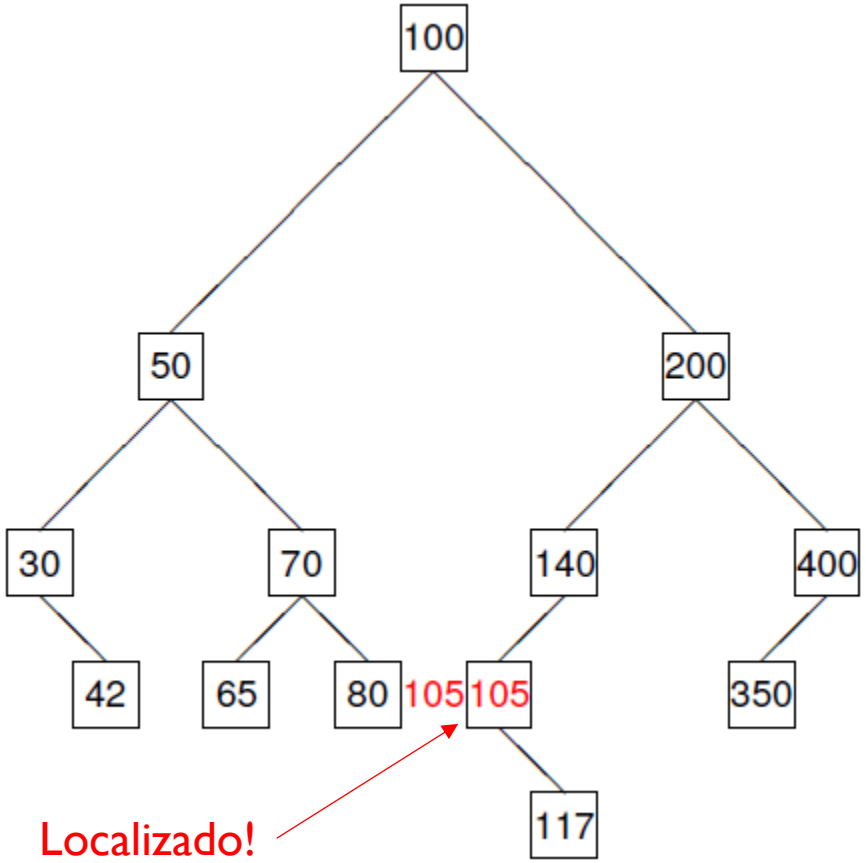
BUSCA

- Dado um valor *x*, deseja-se localizar, se existir, um nó na árvore binária de busca cujo *key* seja igual a *x*.
- A busca começa pelo nó raiz. Quando a busca chega a um nó qualquer da árvore, ou esse nó já contém o valor procurado e a busca termina, ou ele contém um valor menor ou maior que *x*. Isso orienta o prosseguimento da busca em apenas uma das subárvores, podendo descartar a outra subárvore.

PROF. RICARDO MESQUITA

ALGORITMO DE BUSCA

```
function loc(x:integer; t:ref):ref;
var found: boolean;
begin
    found:=false;
    while (t ≠ nil) and not found do
    begin
        if x = t∧.key then
            found:=true
        else
        if x < t∧.key then
            t:=t∧.left
        else
            t:=t∧.right
    end;
    loc:=t
end
```



PROF. RICARDO MESQUITA

COMPLEXIDADE DA BUSCA

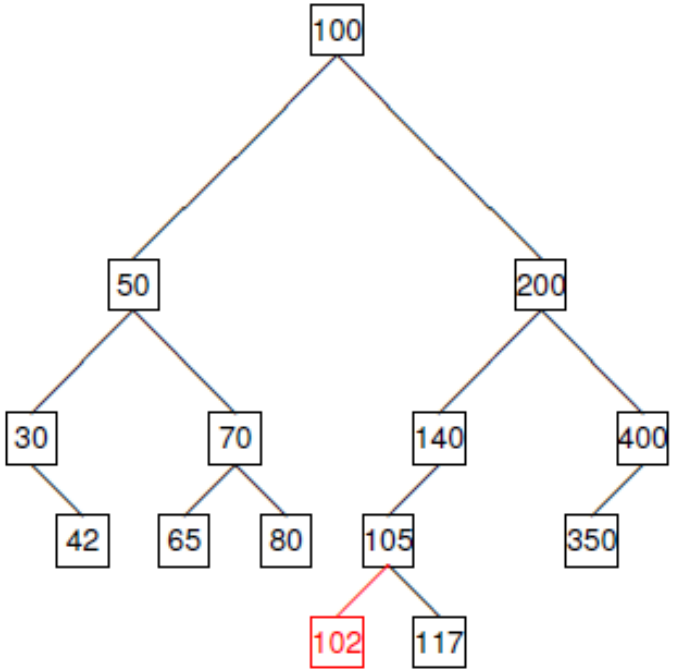
- Dado um valor x, deseja-se localizar, se existir, um nó na árvore binária de busca cujo *key* seja igual a *x*.
- A complexidade do pior caso é igual à altura da árvore.
  - O pior caso pode ser O(n), onde n é o número de nós da árvore.
  - Quando a árvore está ***balanceada***, a busca é eficiente e o pior caso é O(log n).

PROF. RICARDO MESQUITA

INSERÇÃO

- Deseja-se inserir um novo elemento *x* numa árvore binária de busca.
  1. Busca-se x na árvore.
  2. Caso não esteja presente, chega-se a um ponteiro *null* e insere-se o novo elemento nesse lugar.

Exemplo:
Inserir o valor 102 na árvore do exemplo anterior.



PROF. RICARDO MESQUITA

ALGORITMO DE INSERÇÃO

```
type ref= ∧node;
    node = record
        key:integer;
        count:integer;
        left, right:ref
    end;
```

```
procedure insert(x:integer; var p:ref);
{note a chamada por referência do parâmetro p}
begin
    if p=nil then
    begin {não está árvore, então inserir}
        new(p);
        begin
            p∧.key:=x;
            p∧.count:=1;
            p∧.left:=nil;
            p∧.right:=nil
        end
    end
    else if x<p∧.key then
            insert(x,p∧.left)
        else
        if x>p∧.key then
            insert(x,p∧.right)
        else
            p∧.count:=p∧.count+1
end;
```

PROF. RICARDO MESQUITA

ALGORITMO DE INSERÇÃO

- ***Observação***: Se o elemento a inserir já está na árvore, o algoritmo apresentado incrementa um campo chamado *count* que conta o número de ocorrências desse elemento na árvore.

PROF. RICARDO MESQUITA

COMPLEXIDADE DA INSERÇÃO

- Seja uma árvore binária de busca de n nós e de altura h.
- A inserção precisa localizar o local para fazer a inserção.
- A complexidade de inserção é $O(h)$. No pior caso, a altura de uma árvore binária de busca pode ser $O(n)$.

PROF. RICARDO MESQUITA

EXERCÍCIO

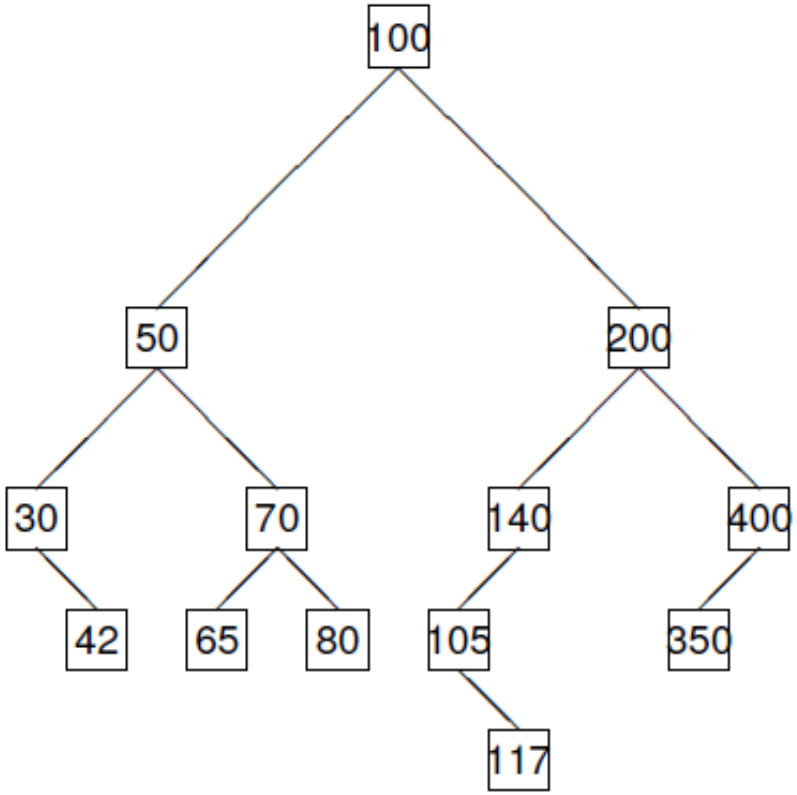
- Os seguintes valores são inseridos, nesta ordem, numa árvore binária de busca inicialmente vazia. Desenhe a árvore resultante.

  100 200 140 50 105 70 80 30 400 350 117 80 42 65

PROF. RICARDO MESQUITA

EXERCÍCIO

Resposta:



PROF. RICARDO MESQUITA

EXERCÍCIO PROPOSTO

- Agora suponha que os seguintes valores são inseridos, nesta ordem, numa árvore binária de busca inicialmente vazia. Desenhe a árvore resultante.

  8 9 11 15 19 20 21 7 3 2 1 5 6 4 13 14 10 12 17 16 18.

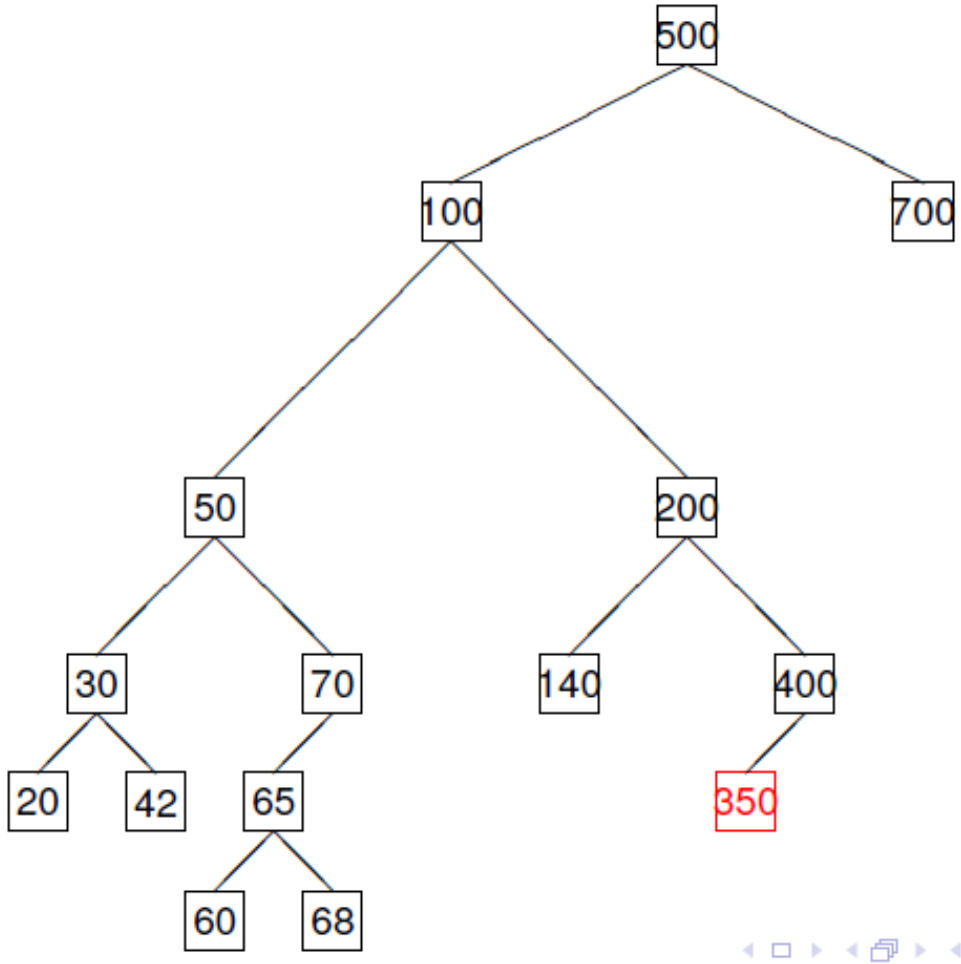
- Se esses mesmos valores forem inseridos numa outra ordem, uma árvore diferente pode resultar? Caso sim, mostre uma ordem de inserção que pode produzir a árvore com menor altura?

REMOÇÃO

- Para remover o nó cujo *key* contém um dado valor *x*, temos que considerar 3 casos:
  1. Não existe nó na árvore com a informação x.
  2. O nó contendo *x* tem 0 ou 1 filho.
  3. O nó contendo *x* tem 2 filhos.

REMOÇÃO

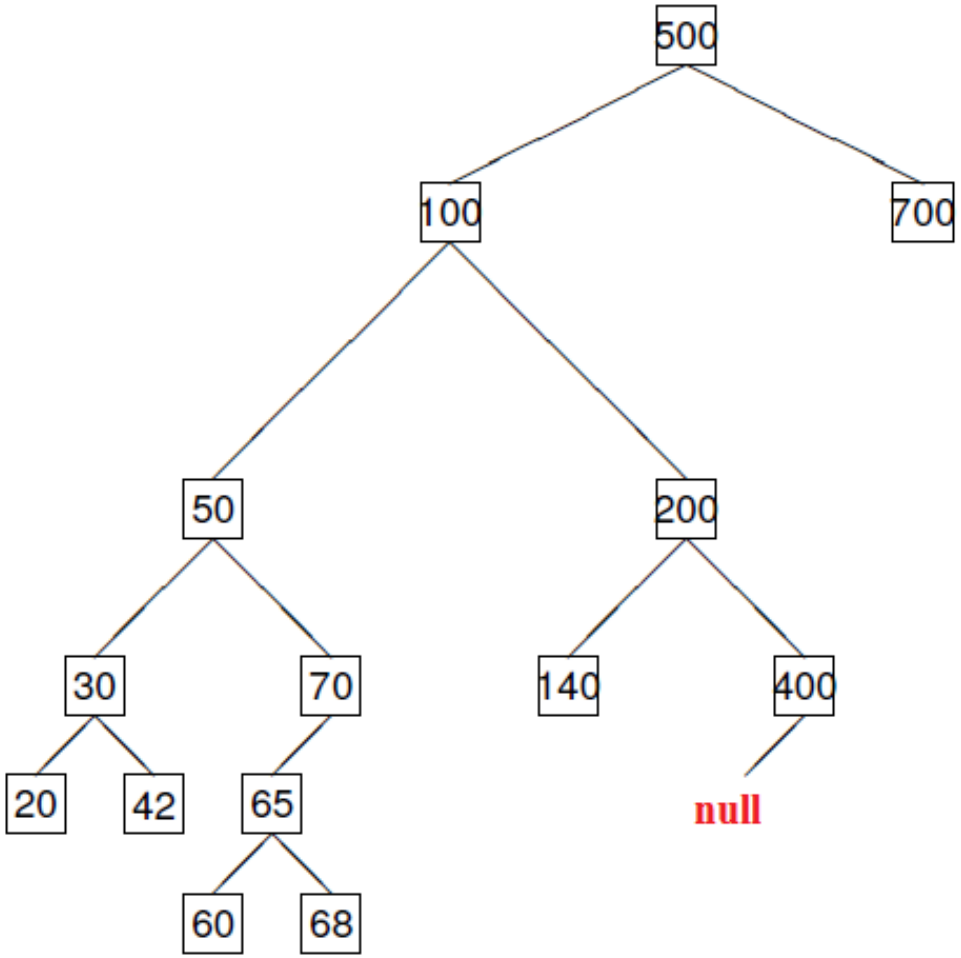
- ***Exemplo***: remover o nó com o valor 350.



PROF. RICARDO MESQUITA

REMOÇÃO

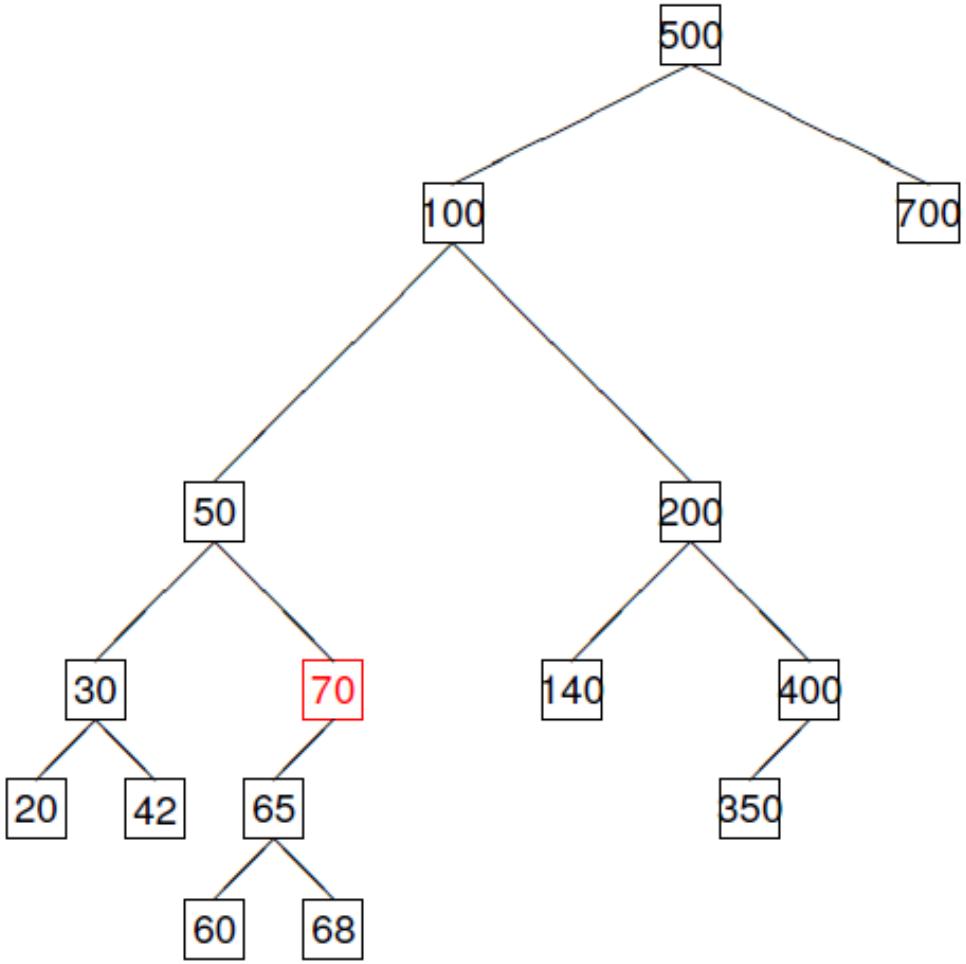
- O nó pai do nó removido passa a apontar para *null*.



PROF. RICARDO MESQUITA

REMOÇÃO

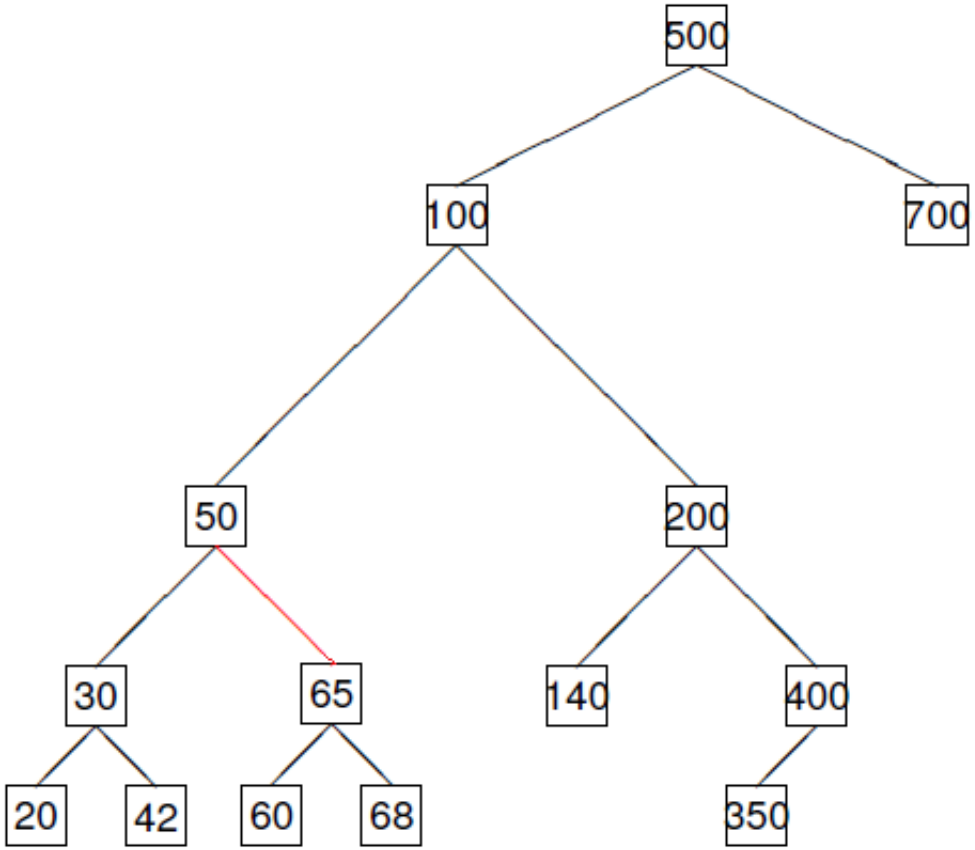
- Exemplo: remover o nó com o valor 70.



PROF. RICARDO MESQUITA

REMOÇÃO

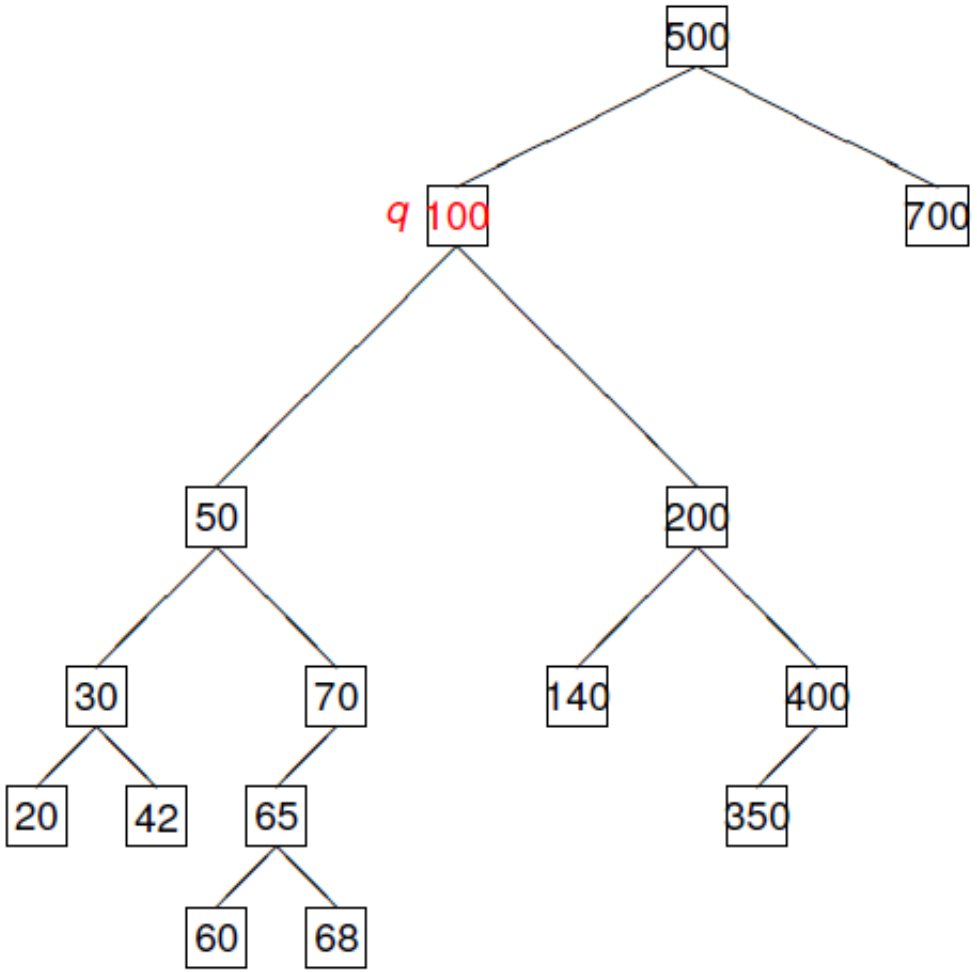
- O pai do nó removido aponta para o filho do nó removido (isto é, neto).



PROF. RICARDO MESQUITA

REMOÇÃO

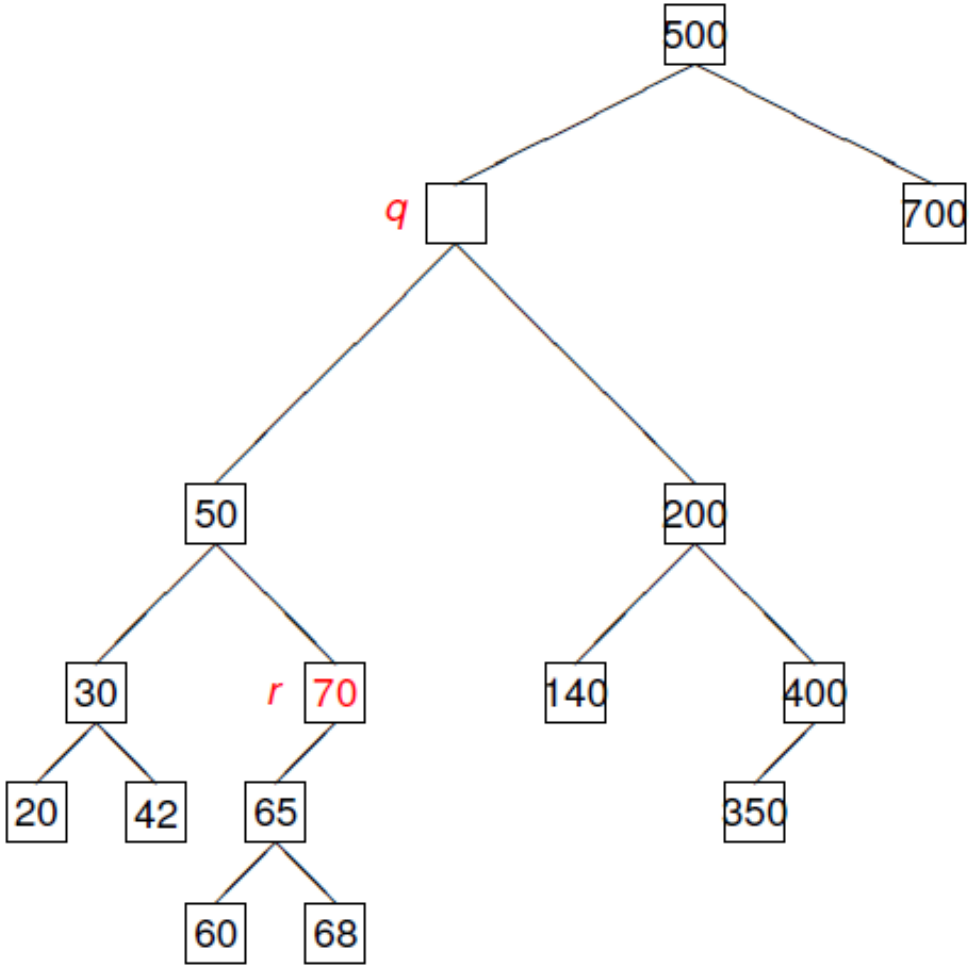
- Exemplo: remover o valor 100 do nó *q*.



PROF. RICARDO MESQUITA

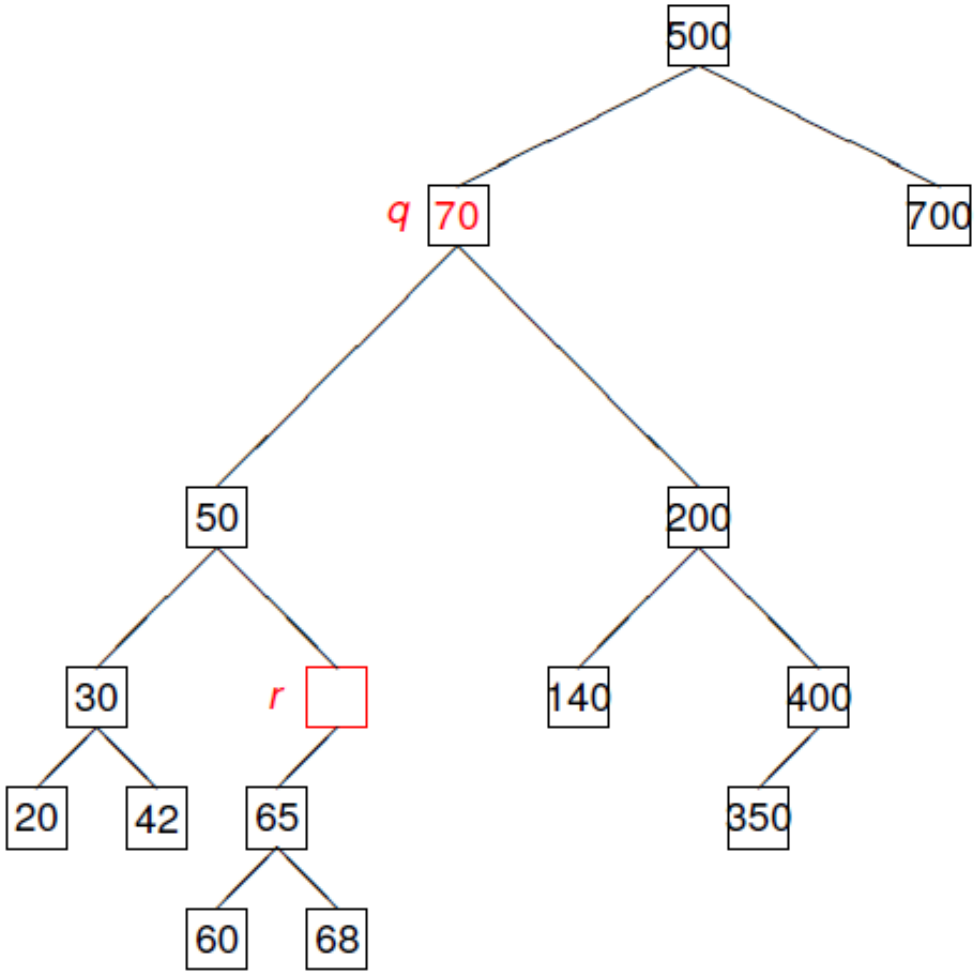
REMOÇÃO

- O valor de q é substituído pelo maior valor (r) da subárvore esq.
- Esse valor (70) está no nó mais à direita da subárvore esquerda.



REMOÇÃO

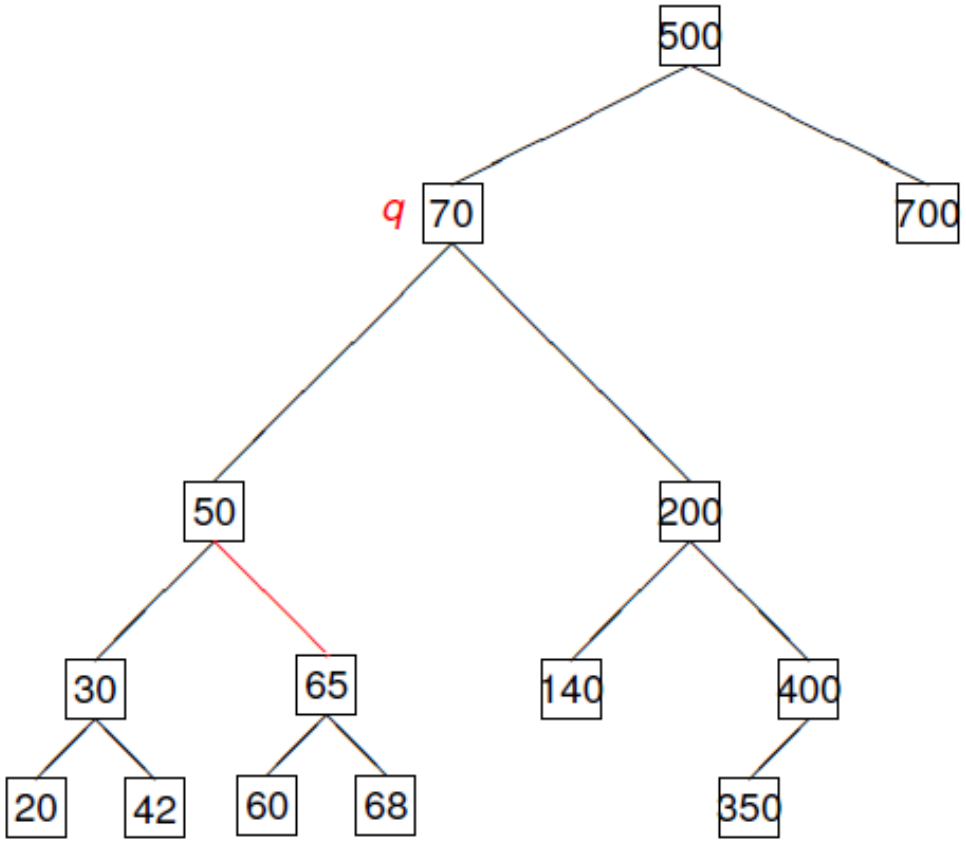
- Agora remove o nó vazio r, que tem um filho só: recai no caso 2.



PROF. RICARDO MESQUITA

REMOÇÃO

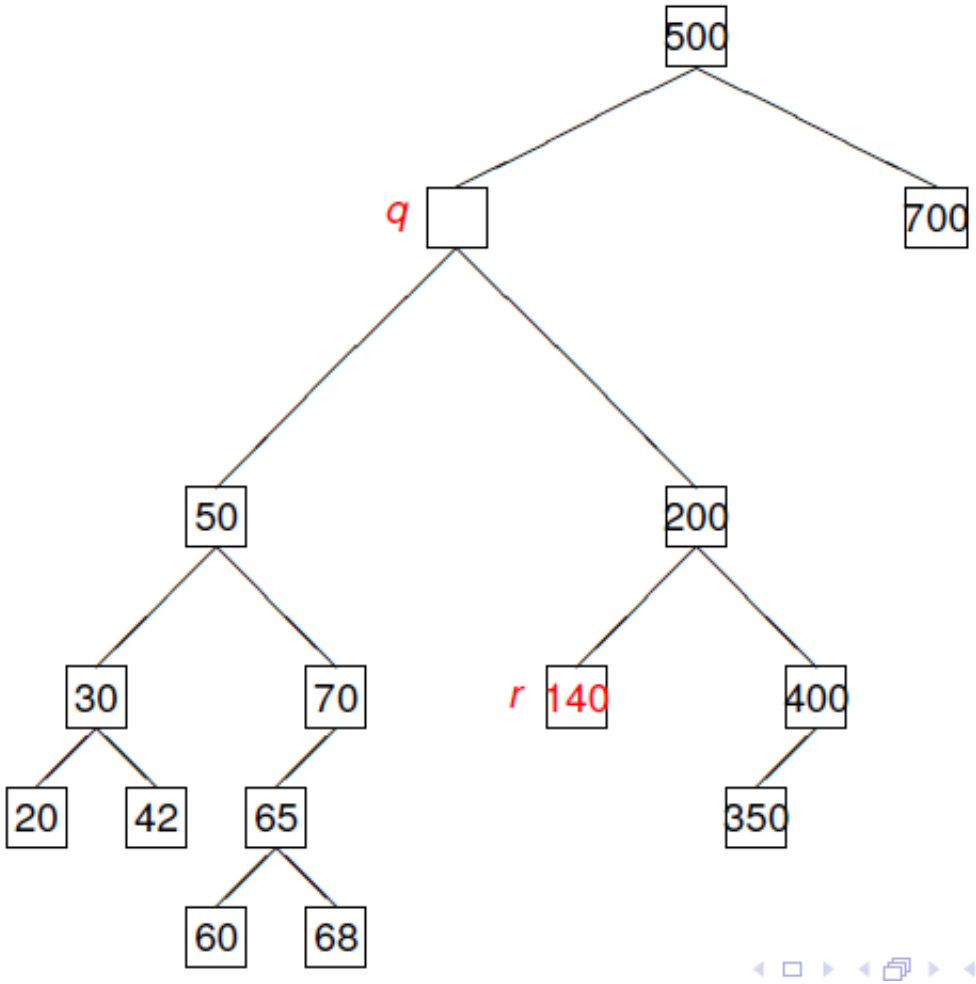
- Basta proceder como no caso 2: o nó removido tem 1 filho.



PROF. RICARDO MESQUITA

REMOÇÃO

- Poderia ter substituído o valor de q pelo menor valor (r) da subárvore direita.
- Esse valor (140) está no nó mais à esquerda da subárvore direita



ALGORITMO DE REMOÇÃO

- O procedimento delete remove o nó contendo x da árvore apontada por p.
- O procedimento auxiliar del cuida do caso 3.
- As variáveis q e r são as mesmas do exemplo ilustrativo.

ALGORITMO DE REMOÇÃO

```
procedure delete(x:integer; var p: ref);
    var q:ref;
    procedure del(var r:ref);
    begin
        if r∧.right ≠ nil then
            del(r∧.right)
        else
        begin
            q∧.key := r∧.key;
            q∧.count := r∧.count;
            q := r;
            r := r∧.left
        end
    end;
```

```
begin {início proc delete}
    if p = nil then
        writeln('Não está na árvore.')
    else
        if x < p∧.key then
            delete(x, p∧.left)
        else
            if x > p∧.key then
            delete(x, p∧.right)
            else
                begin
                    q := p;
                    if q∧.right=nil then
                        p:=q∧.left
                    else if q∧.left=nil then
                        p:=q∧.right
                        else del(q∧.left)
                end;
dispose(q)
end
```

COMPLEXIDADE DA REMOÇÃO

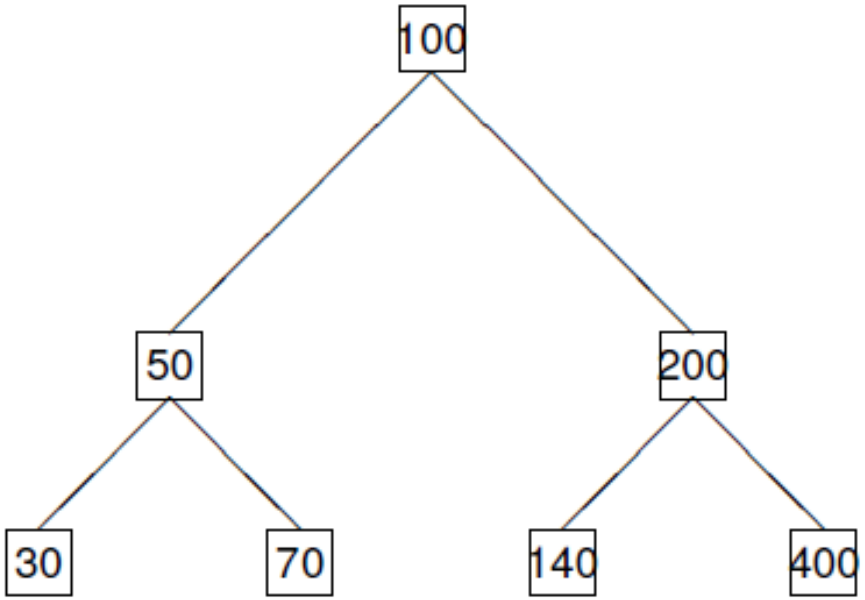
Seja uma árvore binária de busca de n nós e de altura h.

- A remoção precisa primeiro localizar o nó que contém a chave a ser removida.
- Durante a remoção (caso 2) é necessário localizar o nó mais à esquerda (ou mais à direita) de uma subárvore.
- A complexidade de remoção é $O(h)$.
- No pior caso, a altura de uma árvore binária de busca pode ser $O(n)$.

PROF. RICARDO MESQUITA

EXERCÍCIO

- Seja a árvore binária de busca.



- Remova sucessivamente os valores 140, 200, 50 e 100 e desenhe a árvore após cada remoção.

PROF. RICARDO MESQUITA

COMPRIMENTO DO CAMINHO

Comprimento de caminho h_i (*path length*) de um nó i.

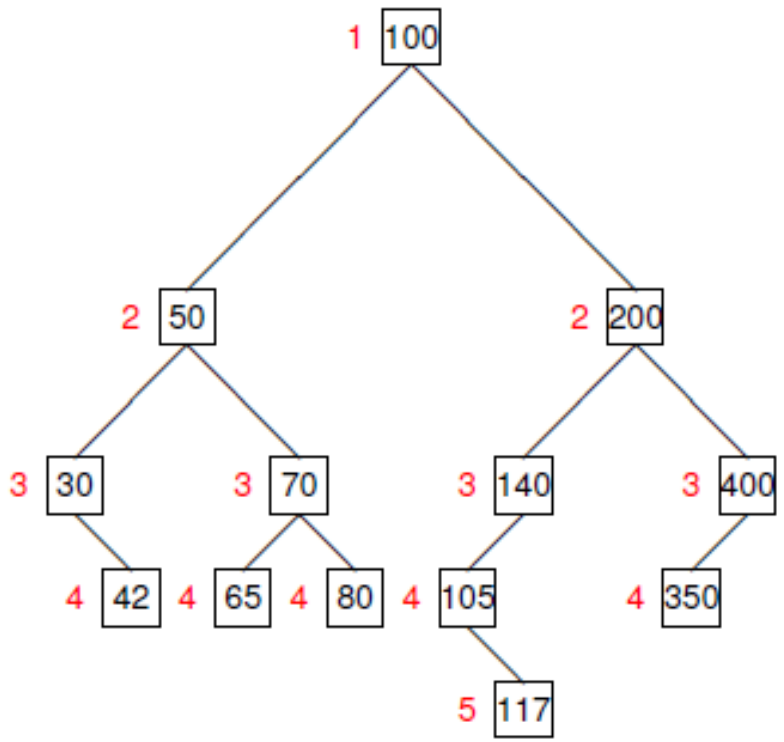
- Se o valor buscado está num nó i da árvore binária de busca, então temos que percorrer e comparar todos os nós desde a raiz até o nó i.
- Denotamos por comprimento de caminho h_i de um nó i o número de **nós** encontrados desde a raiz até o nó i.
- A eficiência de busca de um nó i tem a ver, portanto, com o comprimento de caminho h_i que mede o *número de comparações* realizadas até localizar o nó i.
- *O comprimento de caminho de um nó numa árvore é igual ao nível do nó.*

COMPRIMENTO DO CAMINHO

Comprimento de caminho (path length) de uma árvore binária de busca.

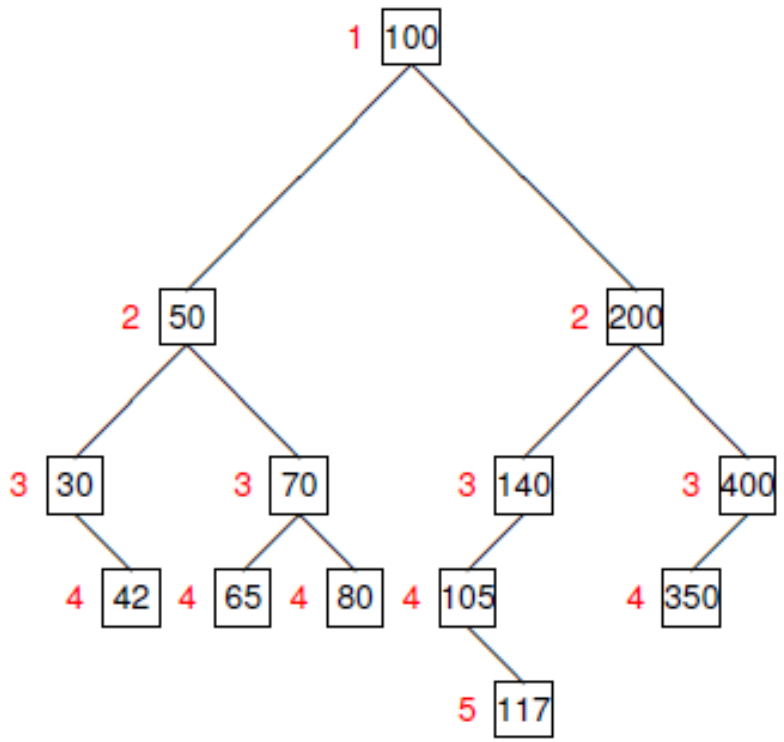
- É a soma dos comprimentos de caminhos de todos os nós da árvore.
- $\Sigma_i h_i$ onde h_i é o comprimento de caminho do nó i.
- Expressa o número total de comparações para se buscar cada nó da árvore.

(Comprimento de caminho da árvore = 42)



COMPRIMENTO DO CAMINHO

- **Comprimento de caminho médio** a_n de uma árvore binária de busca de n nós.
- É o comprimento de caminho da árvore dividido pelo número de nós da árvore: $a_n = (\Sigma_i\ h_i)/n$ onde h_i é o comprimento de caminho do nó i.
- Expressa o número médio de comparações para se buscar um nó da árvore, quando cada nó tem igual chance de ser buscado.
- Comprimento de caminho da árvore = 42
- Comprimento de caminho médio da árvore = 42/13 = 3,2



PROF. RICARDO MESQUITA

ÁRVORES AVL

BALANCEAMENTO

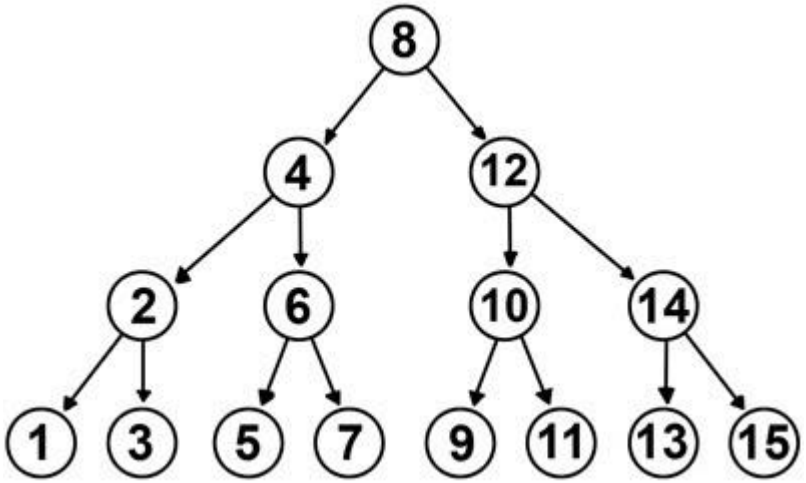
- A altura de uma árvore binária é o nível máximo de suas folhas (profundidade).
- Uma árvore binária é considerada AVL se, e somente se, é uma árvore binária na qual as alturas das duas subárvores (esquerda e direita) nunca difere em mais de 1 unidade.
- O balanceamento de um NÓ é definido como a altura de sua subárvore direita menos a altura de sua subárvore esquerda (Fator de balanceamento ou FB).

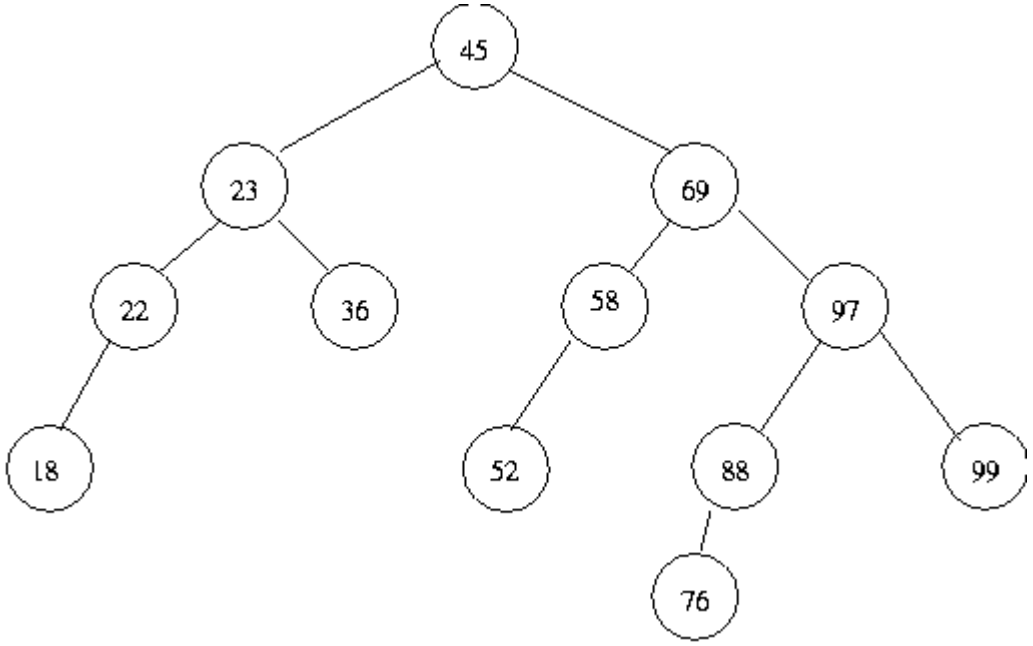
BALANCEAMENTO

- Cada nó numa árvore binária balanceada (AVL) tem balanceamento de +1, -1 ou 0.
- Se o valor do balanceamento do nó for diferente de +1, -1 e 0. Essa árvore não é balanceada (AVL).

ÁRVORE BALANCEADA

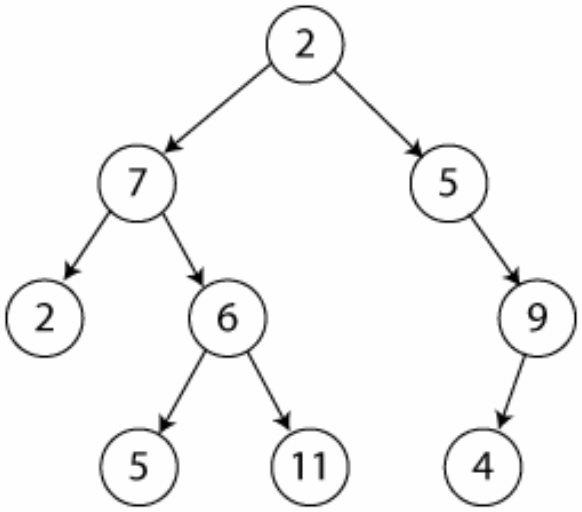
- Exemplo:

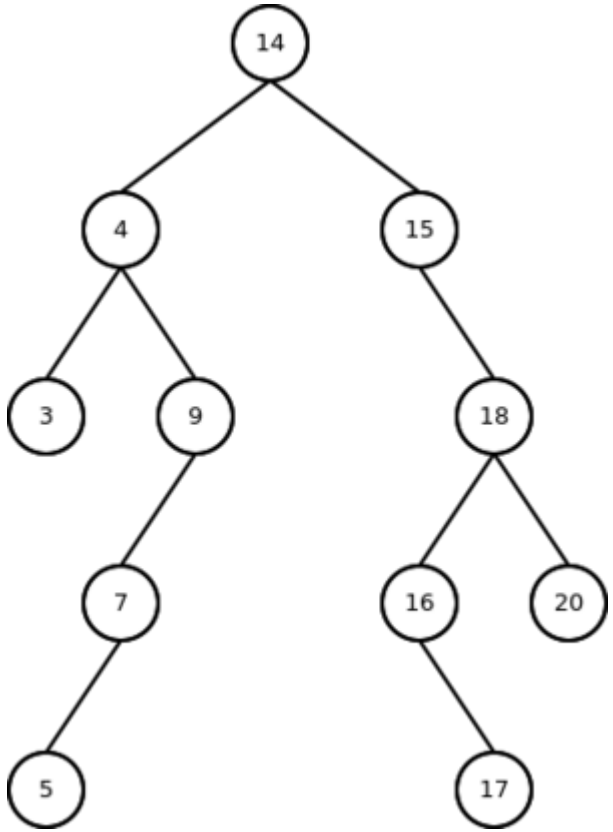




ÁRVORE DESBALANCEADA

- Exemplo:





PROF. RICARDO MESQUITA

POR QUE USAR BALANCEAMENTO?

- É uma estrutura de dados que oferece operações de inserção, remoção e busca em tempo logaritmo, ou seja, é um algoritmo rápido.
- Se a probabilidade de pesquisar um dado for a mesma para todos os dados, uma árvore binária balanceada determinará a busca mais eficiente.
- Os algoritmos vistos até então não garantem que a árvore fique balanceada, assim, o algoritmo da árvore AVL busca deixar a árvore com a menor altura possível.

FATOR DE BALANCEAMENTO

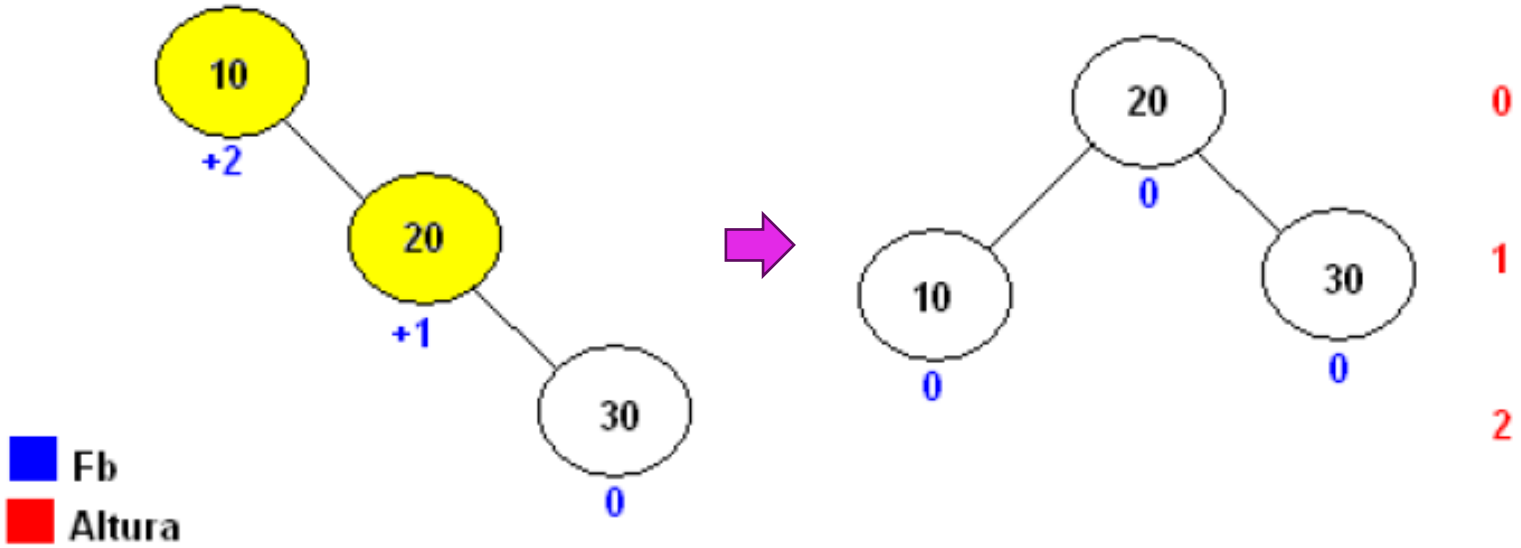
- Para cada nó, define-se um fator de balanceamento (FatBal), que deve ser -1, 0 ou 1. Ele é o responsável por avisar que a árvore está desbalanceada.
- FatBal = altura subárvore direita – altura subárvore esquerda
- FatBal = -1, quando a subárvore da esquerda é um nível mais alto que a direita.
- FatBal = 0, quando as duas subárvores tem a mesma altura.
- FatBal = 1, quando a subárvore da direita é um nível mais alto que a esquerda.
- Toda folha tem FB = 0

QUANDO USAMOS FB?

- Quando inserimos um novo registro na árvore, ou removemos um registro já existente.
- Esta inserção/remoção pode ou não alterar as propriedades de balanceamento.
- Caso a inserção/remoção desse novo registro não viole nenhuma propriedade de balanceamento, podemos continuar inserindo/removendo registros.
- Se a inserção/remoção afetar as propriedades de balanceamento, devemos restaurar o balanço da árvore. Esta restauração é efetuada através de Rotações na árvore.

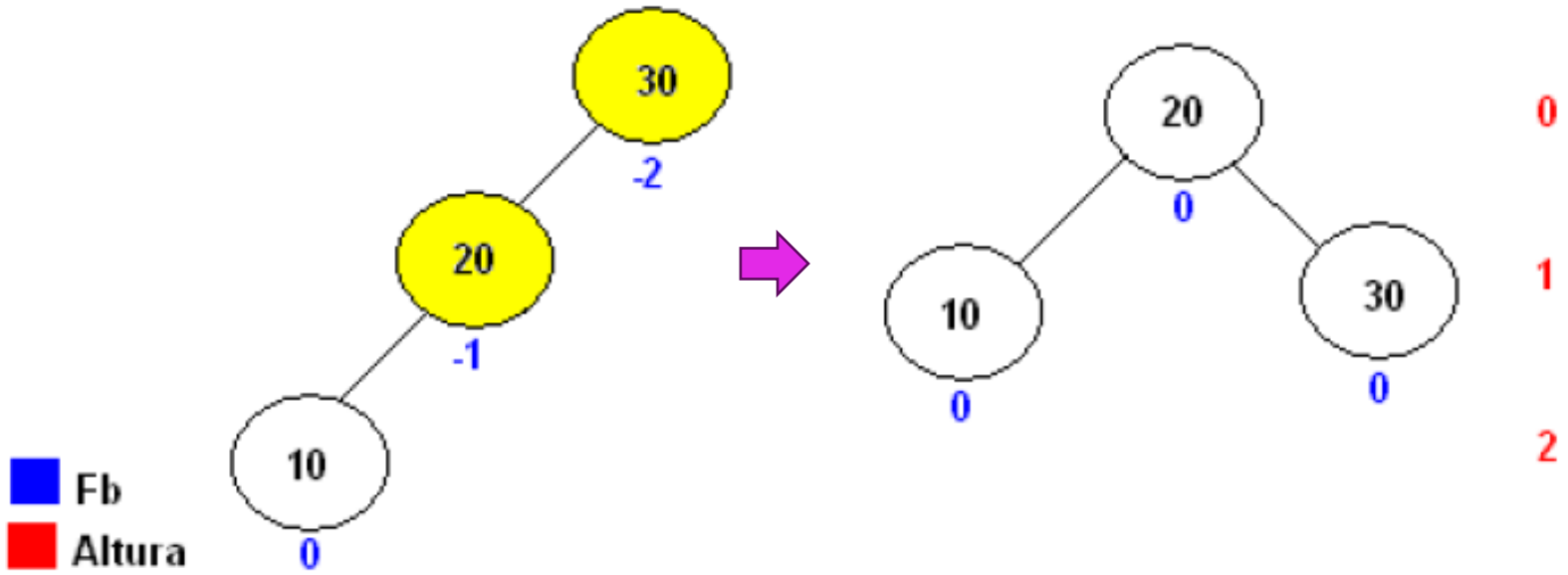
ROTAÇÕES

- Rotação simples à esquerda.



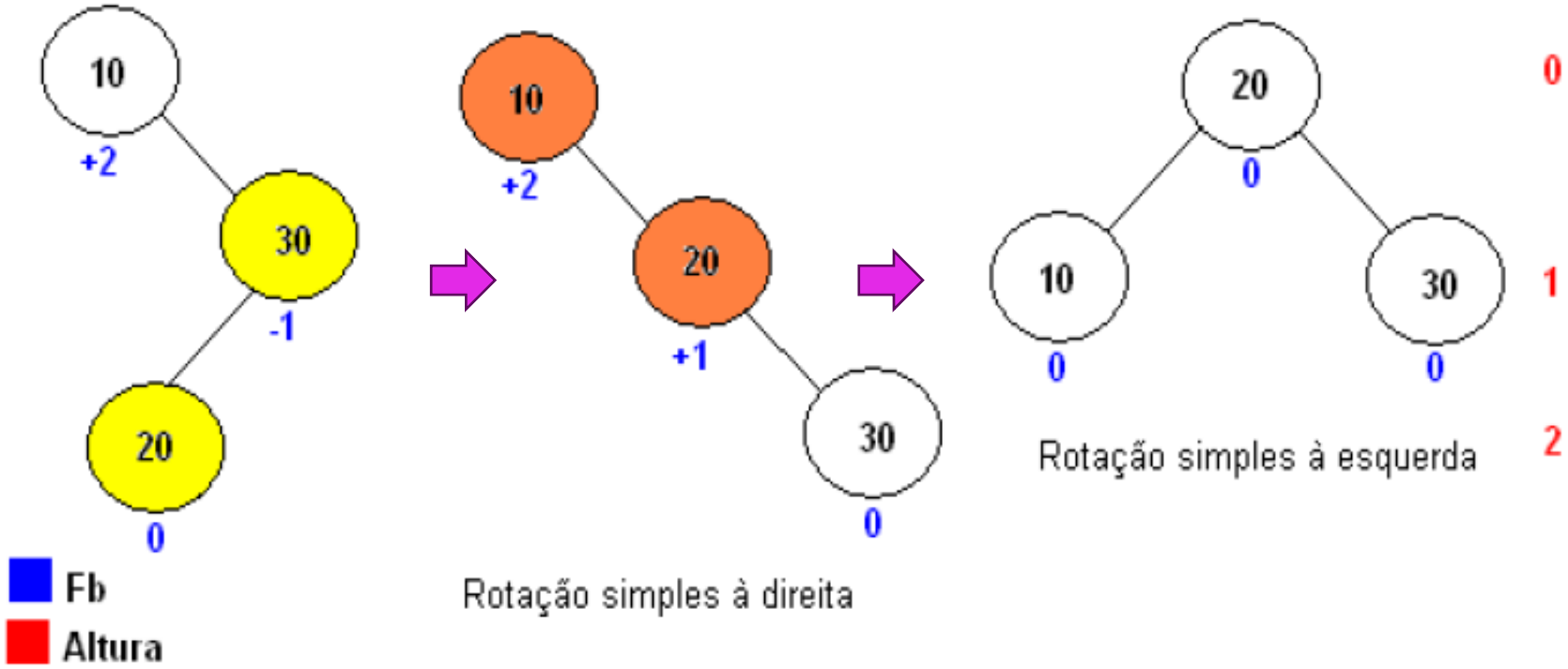
ROTAÇÕES

- Rotação simples à direita.



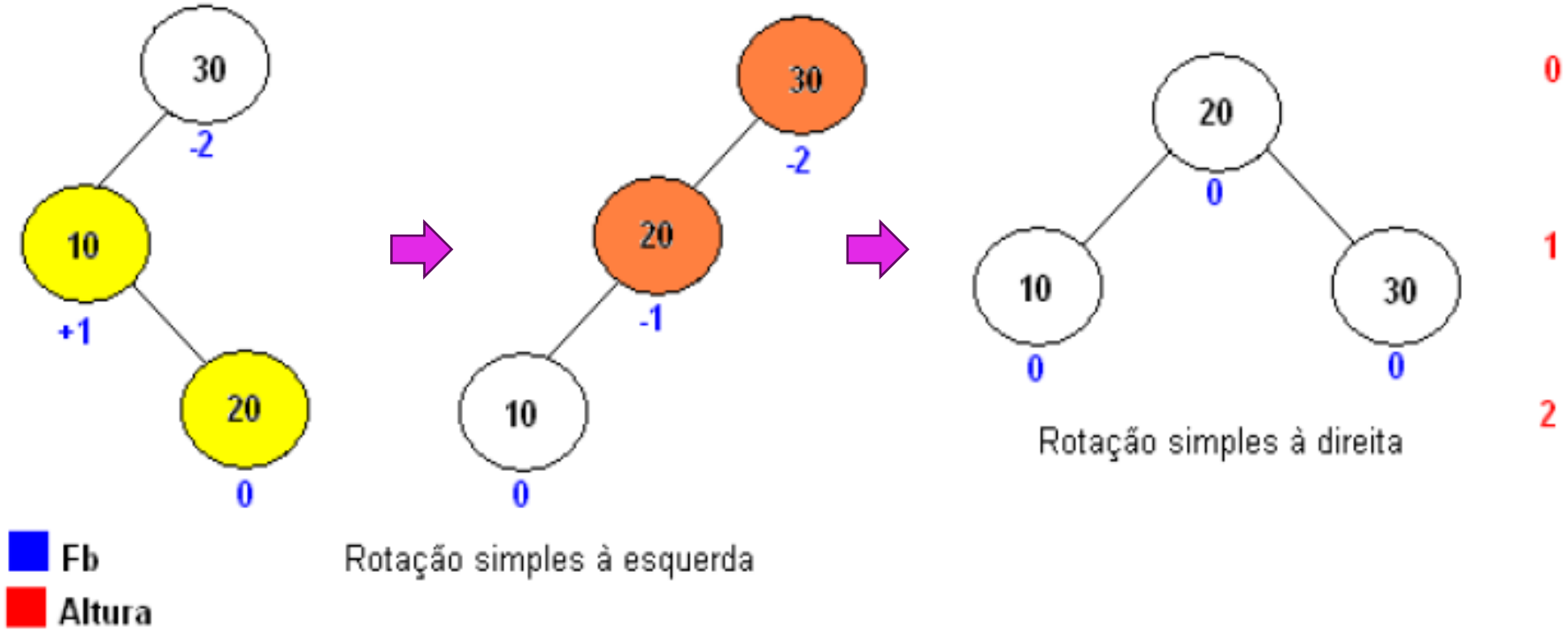
ROTAÇÕES

- Rotação dupla à esquerda.



ROTAÇÕES

- Rotação dupla a direita.



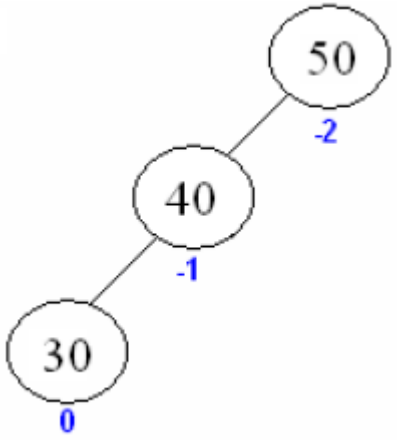
ROTAÇÕES

Dicas

- Para identificar quando uma rotação é simples ou dupla deve-se observar os sinais do FB:
  - Se o sinal for igual, a rotação é simples.
  - Se o sinal for diferente, a rotação é dupla.
- Se FB for positivo (+), a rotação é para à esquerda (sentido anti-horário).
- Se FB for negativo (–), a rotação é para à direita (sentido horário).

ROTAÇÃO SIMPLES

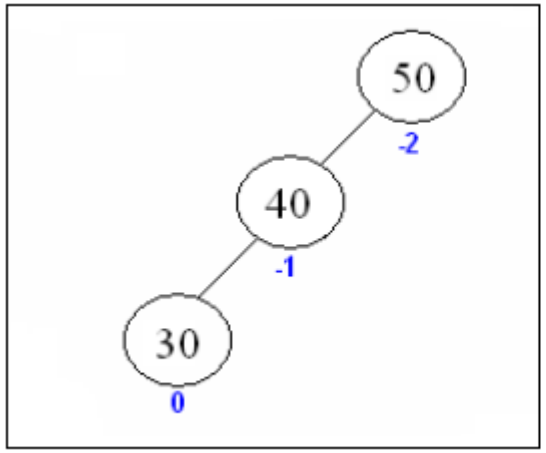
- Suponha que inserimos os números 50, 40 e 30 em uma árvore. Obteremos então:

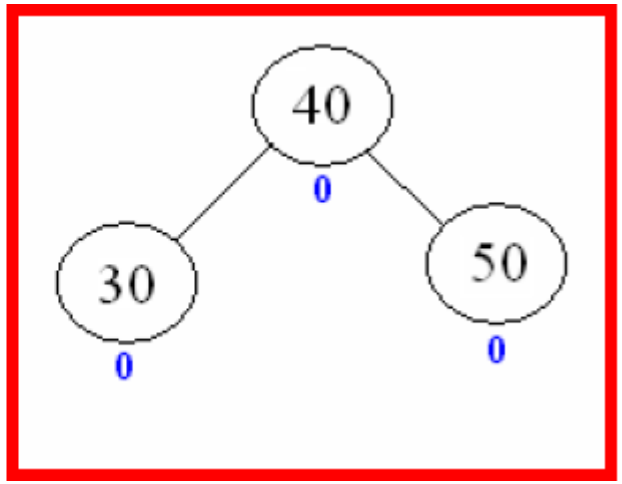


- A inserção produziu um desbalanceamento.
- Neste caso, como os sinais dos FB são os mesmos, significa que precisamos fazer apenas uma ROTAÇÃO SIMPLES à direita no nó com FB -2.
- No caso simétrico (nó com FB 2) faríamos uma rotação simples à esquerda.

ROTAÇÃO SIMPLES

- Após a rotação simples teremos:

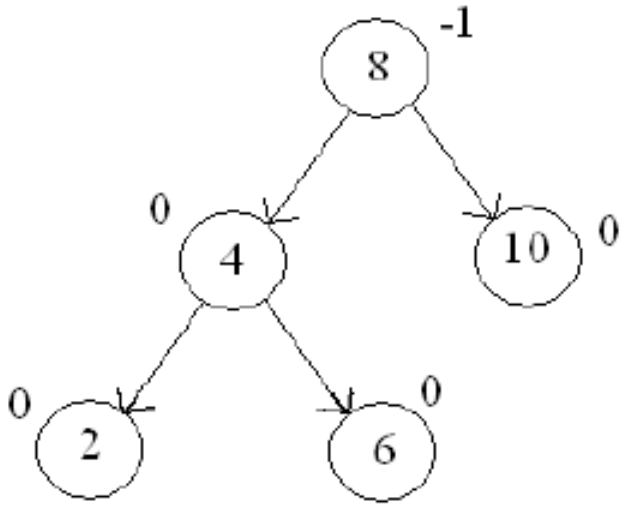




- Agora a árvore está balanceada.

EXEMPLO

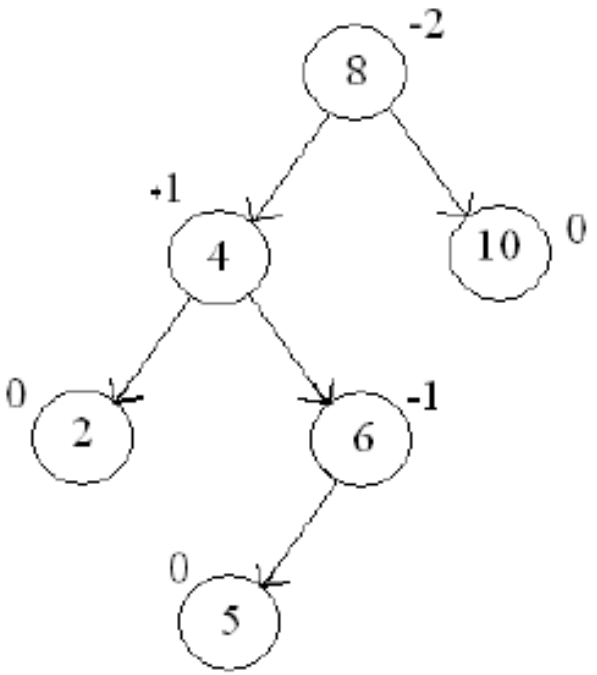
- Considere a árvore ao lado:



- Como podemos observar, a árvore está balanceada pelos FB de cada nó.

EXEMPLO

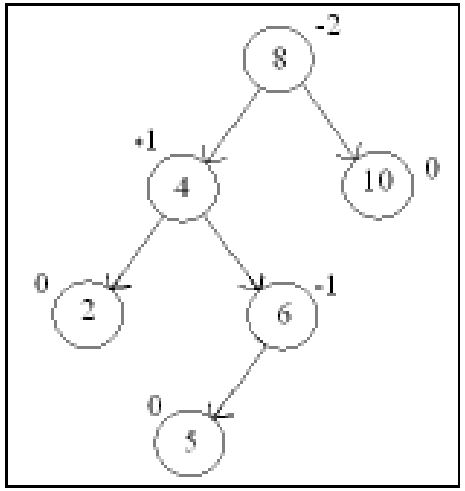
- Ao inserir uma chave 5 na árvore:

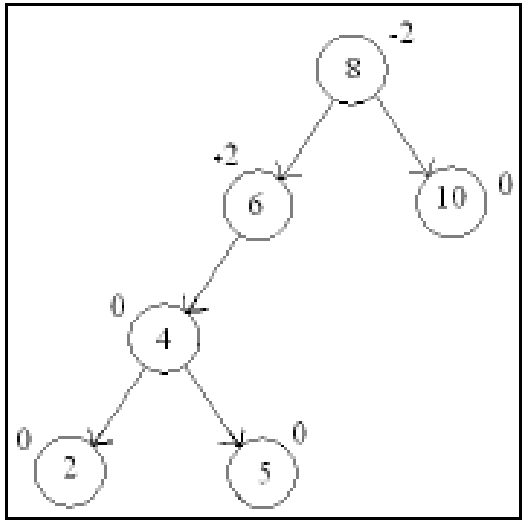


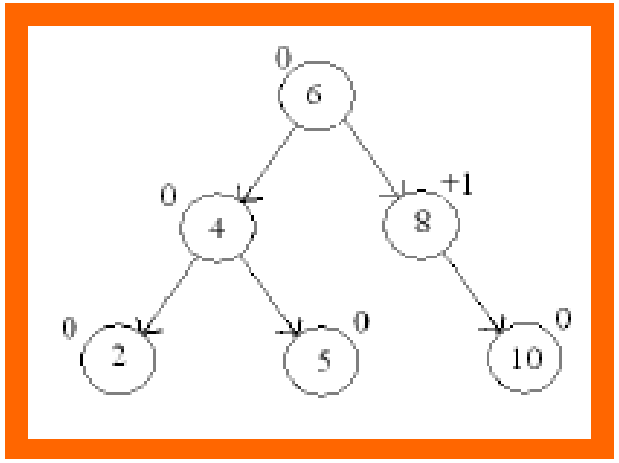
- O nó 8 fica com o FB -2 e tem um filho com FB +1.
- Neste caso para manter o balanceamento devemos aplicar duas rotações, ou seja, uma ROTAÇÃO DUPLA.

EXEMPLO

- Primeiro rotaciona-se o nó com FB 1 para a esquerda.
- Logo rotaciona-se o nó que possuía FB -2 na direção oposta, nesse caso a direita.







EXEMPLO

- Os FB dos nós voltaram a ficar dentro do esperado das árvores AVL.
- O caso simétrico ao explicado acima acontece com os sinais de FB trocados, ou seja, um nó com FB +2 com um filho com FB -1. Também utilizaríamos uma rotação dupla, mas nos sentidos contrários, ou seja, o nó com FB -1 seria rotacionado para a direita e o nó com FB +2 seria rotacionado para a esquerda.

SIMULADOR

- https://www.inf.ufsc.br/~aldo.vw/estruturas/simulador/AVL.html

DÚVIDAS?